Set 5 : https://claude.ai/public/artifacts/83eee27a-0509-45c1-be08-cc3e8b4cbfc8

A capability is purely a set of written instructions and reference examples for how to format a specific report, used only within Claude Code by one team, with no external system to call. Which approach fits, and why not an MCP server?

- ○ Hard-coded into CLAUDE.md with no other structure, since it's team-specific
- ○ A custom tool, since tools are the default choice whenever multiple people will use something
- ○ An MCP server, because any reusable capability should default to a server regardless of whether it calls anything external
- ✓ A Skill; there is no external capability or live system to expose, so a server would add operational overhead for no benefit

Correct

Instructions and reference material with nothing external to call is the textbook Skill case; standing up a server here adds cost with no corresponding capability gained. Tools represent actions rather than formatting guidance, and while CLAUDE.md could hold some of this, a Skill is the structured, reusable form built for exactly this need.

A 2,000-line 'do everything' prompt-handling function has become unreadable and every change risks breaking something else. What is the appropriate response?

- Leave it alone since prompt-handling code is inherently unstructurable
- Add a large explanatory comment at the top and continue extending the function as before
- ✓ Refactor incrementally into smaller, well-named functions with clear responsibilities, backed by tests, rather than rewriting from scratch
- Rewrite it entirely from a blank file in one sitting with no tests

Correct

Incremental, tested refactoring reduces risk while improving structure, which is the standard response to a growing single-responsibility violation. A full rewrite with no tests risks losing behaviour, a comment does not fix the structural problem, and 'it can't be structured' is simply false.

Which TWO are recognised agent design patterns for handling multi-step tasks?

Select 2 · chosen 2

- [ ] Disabling all tools to force the model to reason purely from its own knowledge
- [ ] Hard-coding every possible input as its own branch so no runtime decision is ever needed
- [x] Delegating a bounded subtask to a subagent that returns a compact result
- [x] A tool-use loop where the agent repeatedly calls tools and incorporates results until done

Correct

The tool-use loop and subagent delegation are established agent patterns for multi-step work. Branching every possible input by hand is the workflow approach taken to an unworkable extreme, and disabling tools removes the agentic capability rather than patterning it.

Your content policy is enforced only by a single line in the system prompt. What is the recommended improvement per secure-by-design principles?

- ✓ Layer independent enforcement: policy guidance in the prompt plus output-side filtering plus scoped tool permissions, so no single control is a single point of failure
- Make the system-prompt line longer and more emphatic, since a sufficiently detailed instruction is an adequate standalone control
- Remove the system-prompt line since it adds no value on its own
- Rely on the user to self-police what they ask for

Correct

Secure-by-design calls for layered, independent controls rather than one probabilistic instruction. A longer prompt line is still a single, bypassable layer, removing it entirely drops even that layer, and user self-policing is not a control at all.

The same feature must work inside claude.ai, through the API, and inside Claude Code. A teammate assumes one well-written prompt will behave identically everywhere. What's the flaw?

- There is no flaw; a single prompt is guaranteed to behave identically across every interface
- ✓ Each surface wraps the prompt in different default context and instruction placement, so identical wording can still behave differently across them
- Claude Code ignores system prompts entirely
- Only the API supports system prompts

Correct

Different interfaces layer different default context and instruction handling around the same words, so behaviour can diverge. The API does support system prompts and Claude Code does not ignore them, and no interface guarantees identical behaviour by default.

A capability needs to call three different internal REST services with complex auth, be reused across five separate Claude applications, and be maintained by a dedicated platform team. Which approach fits best?

- Five separate hand-written custom tools, one per application, kept manually in sync
- ✓ An MCP server, maintained centrally and connected to by each of the five applications
- One shared, hardcoded block of instructions pasted into each application's system prompt
- A Skill duplicated into each of the five applications' repositories

Correct

Centralized maintenance, complex auth, and reuse across many applications is exactly the MCP server case. Duplicating a Skill or hand-syncing five custom tools both create maintenance burden the centralized server avoids, and pasted instructions cannot perform authenticated REST calls at all.

A stakeholder says 'the bot should understand customer intent.' Before building anything, what is the right next step?

✗ Write the system prompt first and derive requirements from what it ends up doing

Pick the largest model so intent understanding is as strong as possible regardless of scope

Start building immediately and let the model's general capability handle whatever 'intent' turns out to mean

✓ Translate this into concrete functional requirements: which intents, what fields to extract, and what counts as a correct classification

Not quite

Vague business language must become checkable functional requirements before design starts. Building first, oversizing the model, or reverse-engineering requirements from a prompt all skip the step that makes the system buildable and testable.

You are designing a tool set for an agent and want to minimise wrong-tool selection from the start, before any bugs appear in production. Which practice addresses this proactively?

- Give every tool an identical, generic description so the agent isn't biased toward any one of them
- Rely on the model's general judgement and skip writing descriptions with much detail
- ✓ Write each tool's description to be mutually distinct, including when not to use it, and keep the tool set focused rather than sprawling
- Add as many tools as possible upfront so the agent has maximum flexibility for any future task

Correct

Distinct, well-scoped descriptions with explicit boundaries are the proactive defense against wrong-tool selection. A sprawling tool set or generic descriptions both increase ambiguity, and skipping detail leaves the model with nothing to disambiguate on.

An assistant confidently states a statistic that sounds authoritative but cannot be traced to any source you gave it. What practice would have caught this before it reached the user?

- Raising the temperature, since more varied phrasing tends to self-correct factual errors
- ✓ Skepticism toward confident output: validate specific claims against your source material rather than trusting fluent, confident phrasing
- Nothing; if the phrasing is confident and fluent, the content is reliable
- Switching to a smaller model, which hallucinates less than larger ones

Correct

Confident, fluent phrasing is not evidence of accuracy, so specific claims need validation against real sources. Trusting fluency is exactly the failure mode, temperature does not fix factual grounding, and model size alone does not guarantee lower hallucination rates.

A request reaches your Claude-powered admin panel with a valid session cookie. Before letting it delete a user account, what must you additionally verify?

- ○ Only that the model itself agrees the action seems reasonable
- ✓ Authorization: that this specific authenticated identity is permitted to perform this specific action, not merely that they are logged in at all
- ○ Only that the request is over HTTPS
- ○ Nothing further; a valid session cookie is sufficient proof the action should proceed

Correct

Authentication proves who someone is; authorization separately checks whether that identity may perform this specific action. A valid session alone does not establish permission, HTTPS protects transport not permission, and the model's own judgement is not an access control.

You invoke Claude through a third-party vendor's hosted endpoint rather than the first-party API. Which statement is accurate?

- Third-party vendors run a completely different model that merely shares the same name
- ✓ The Messages API request and response shape stays consistent, but authentication, available headers, and some access patterns can differ by vendor
- Tool use is unavailable when invoked through a third-party vendor
- Streaming is only available through the first-party API

Correct

Vendor-hosted access generally preserves the Messages API shape while differing in auth and some platform-specific details. It is the same model family, and streaming and tool use are not first-party exclusives.

A legal-document summarizer must catch subtle contractual nuance, and an internal wiki search bot just needs to find the right page fast. How should tiers differ?

- Use the same tier for both, since consistency across features matters more than matching capability to task difficulty
- Use the cheapest tier for both to control cost uniformly
- ✓ Use a higher-capability tier for the nuanced legal task and a faster, cheaper tier for the simpler retrieval-style task
- Use the highest tier for both to avoid any risk of missing something

Correct

Matching tier to task difficulty is the core selection tradeoff: nuance-heavy work justifies a higher tier, and simple retrieval does not need it. Uniform tier choice in either direction either overpays or underserves one of the two tasks.

Which TWO of these are examples of output constraints, as distinct from few-shot examples?

Select 2 · chosen 2

- [ ] Explaining the company's mission statement for context
- [x] ✓ Specifying a maximum word count and a required set of JSON fields
- [x] ✗ Showing three worked input-output pairs before the real task
- [x] ✓ Requiring the response to omit any preamble and start directly with the answer

Not quite

A word limit and required field set, plus a no-preamble rule, are direct constraints on the shape of the output. Worked pairs are few-shot examples rather than constraints, and a mission-statement explanation is background context, not a constraint on output form.

A Claude feature has shipped and is live. Which activity belongs to the operate-and-maintain phase of the life cycle, not an earlier phase?

- Writing the initial functional requirements
- ✓ Monitoring production quality and cost, and triaging regressions as the model or usage pattern shifts
- Selecting which model tier to prototype with
- Designing the eval suite for the first release

Correct

Operate-and-maintain is about post-launch monitoring and response to drift or regressions. Requirements, tier selection, and eval design happen earlier, during design and build.

A plugin your app depends on requires plugin B at version 2.x, but you've pinned plugin B at 1.x elsewhere in the project. What kind of problem is this?

- A harmless duplication, since plugins do not share state and version numbers are cosmetic
- Something only the model can resolve at runtime by picking whichever version seems newer
- ✓ A plugin dependency conflict that must be resolved by aligning versions or isolating the plugins before either can be trusted in production
- A pure performance issue that only affects load time

Correct

Conflicting version requirements across plugins is a dependency conflict that needs explicit resolution, exactly like any software dependency graph. It is not cosmetic, not purely a performance concern, and not something the model silently arbitrates.

Your API client currently does `json.loads(response)` with no try/except. What is the minimum defensive-parsing improvement?

- Nothing needs to change as long as the prompt asks for JSON
- ✓ Wrap the parse in error handling, validate the parsed structure against an expected schema, and define a fallback for malformed output
- Increase max_tokens so the JSON is less likely to be malformed
- Add more few-shot examples of JSON output only, with no code-level changes at all

Correct

Defensive parsing means handling the error, validating structure, and defining a fallback in code, since a prompt request alone does not guarantee conformance. max_tokens does not guarantee valid syntax, and examples help but do not substitute for code-level defense.

You're deciding between running an agent yourself on your own infrastructure versus using an Anthropic-hosted managed agent deployment. What's the core tradeoff?

- There is no meaningful difference; both options run identically with identical operational responsibilities
- ✓ Self-hosting gives you full control over infrastructure and data flow at the cost of operating it yourself; a managed deployment reduces operational burden at the cost of less infrastructure control
- Managed deployment is only available for non-agentic use cases
- Self-hosting is always cheaper regardless of the team's operational capacity

Correct

The tradeoff is control and operational burden versus convenience: self-hosting means you own the infrastructure, managed hosting shifts that operational load elsewhere. The two are not equivalent, cost depends on context rather than being universally lower for self-hosting, and managed deployment is squarely an agentic-use option.

Your team wants to know, after the fact, exactly which service account accessed a sensitive internal tool and when. What capability does this require?

- A single shared service account for simplicity
- ✓ Authorized access monitoring and logging tied to identity, so actions can be attributed and reviewed after the fact
- A larger context window, so the model remembers who called it
- Prompt caching on the tool's description

Correct

After-the-fact attribution requires identity-tied access logging and monitoring. Context window size has nothing to do with audit trails, a single shared account destroys attributability, and caching a description is unrelated to access logs.

A tool call fails because the downstream service is temporarily down. What should the tool return to the agent?

- The same generic error message regardless of whether the failure is temporary or permanent
- A silently truncated partial result with no indication anything went wrong
- ✓ A structured error indicating the failure is transient and retriable, distinct from a permanent or invalid-input failure
- An empty string with no further information

Correct

A structured, categorised error lets the agent decide to retry, wait, or escalate appropriately. An empty string or generic message gives no actionable signal, and a silent partial result actively misleads the agent into treating a failure as success.

A dashboard needs to show tokens streaming in over a persistent connection, with the server able to push updates without the client re-requesting. Which underlying technology is being described?

- The Message Batches API
- ✓ A websocket, which keeps a persistent bidirectional connection open
- A CDN cache
- A one-off synchronous REST call repeated on a polling interval

Correct

A websocket provides the persistent, bidirectional connection needed for server-pushed streaming updates. Polling re-requests repeatedly rather than staying open, Batches is asynchronous and non-live, and a CDN cache serves static content rather than a live stream.

Two independent subagents each need a large shared reference document to complete their separate parts of a task, but the documents should not pollute each other's follow-up reasoning with the other's intermediate steps.

- Give the reference document to only one subagent and have it verbally summarise the whole thing to the other
- ✓ Give each subagent its own isolated context containing the shared reference, so their intermediate reasoning never crosses over
- Skip giving the document to either and let both reason from general knowledge instead
- Run both subagents in a single shared context so they can see each other's reasoning as it happens

Correct

Context isolation through separate subagent contexts lets each hold the shared reference without their intermediate reasoning bleeding together. A single shared context defeats the isolation goal, one subagent narrating to the other loses fidelity, and skipping the reference entirely abandons the actual source material.

You want a guarantee that a specific shell command can never run, no matter what the agent decides mid-session. Which mechanism actually provides a guarantee, as opposed to a strong suggestion?

- Setting a lower temperature for that session
- A detailed system-prompt paragraph forbidding that command
- ✓ A hook that deterministically intercepts and blocks the command before execution
- Choosing a model known for being cautious

Correct

A hook enforces deterministically at the point of execution, which is what a guarantee requires. A prompt instruction, a cautious model, or a lower temperature are all probabilistic and can be overridden by circumstances the guidance did not anticipate.

A task's steps are mostly fixed, but one step occasionally needs the agent to choose between two valid approaches based on data it hasn't seen before. How should this be architected?

- Two entirely separate hard-coded workflows, one per possible approach, chosen at random
- A rigid workflow with no agentic step, requiring a human to manually pick the approach every time
- ✗ A fully autonomous agent for every step, since any variability anywhere means the whole task should be agentic
- ✓ A workflow overall, with a small bounded agentic decision point only at the one variable step

Not quite

Mostly-fixed steps with one point of genuine judgement call for a workflow with a small bounded agentic decision, not a wholesale switch to autonomy. Full autonomy overcorrects for one variable point, a fully rigid workflow can't make the judgement call at all, and random selection ignores the data meant to inform the choice.

Your app pins claude-model-2026-06-15 in production. A new dated version is released with a documented breaking change to how it formats numbered lists. Which TWO are correct next steps?

Select 2 · chosen 2

- [ ] Ignore the release notes since breaking changes only apply to floating aliases, not dated versions
- [ ] Upgrade immediately in production, since a dated release is always safe to adopt without testing
- [x] Test the new version against your eval suite and downstream parser before promoting it
- [x] Read the release notes for the specific breaking change and check whether your prompt or parser depends on the old formatting

Correct

A documented breaking change demands checking your specific dependency on the old behaviour and validating the new version against evals before promoting it. Dated versions are not automatically safe to adopt blind, and breaking changes affect anyone consuming that version, not just floating aliases.

A prompt buries the single most important instruction in the middle of six paragraphs of background. The model keeps missing it. What is the most direct fix?

- [x] Move the critical instruction to the beginning or end of the prompt, where models attend most reliably, and trim unnecessary background
- [ ] Repeat the six paragraphs of background twice so the instruction gets more total exposure
- [ ] Switch to a lower temperature so the model reads more carefully
- [ ] Add the instruction as a sixth occurrence in the middle so it appears more often

Correct

Long inputs are read less reliably in the middle, so critical instructions belong at the start or end, with unneeded background trimmed. Doubling background or repeating mid-document placement does not fix the position problem, and temperature does not affect positional attention.

An MCP server exposes a large static company glossary that Claude should be able to read into context when relevant, without the model needing to invoke an action to fetch it each time. Which MCP primitive fits best?

- A transport setting
- ✓ A resource
- A prompt template
- A tool, since any content the model uses must be fetched through an explicit tool call

Correct

Resources are the MCP primitive for exposing readable reference data the model can pull into context. Tools represent actions rather than passive reference data, prompts are user-invoked templates, and a transport is the communication channel, not content.

Two engineers each add a Claude-calling feature to the same file over the same sprint with no code review step. What SDLC practice is missing?

- ✓ Code review integrated into the merge process, catching conflicting assumptions before they reach production
- Prompt caching, which prevents merge conflicts
- A more capable model, which would have resolved the conflicting assumptions on its own
- A larger context window, which would have let one engineer see the other's changes automatically

Correct

Code review is the SDLC step that catches conflicting assumptions before merge; skipping it is exactly what let two independent changes collide. Context window size, model capability, and caching do not address a missing human review step.

A colleague claims 'zero-shot' means the model has zero context about the task. What's the accurate description?

- Zero-shot means the model ignores the user's question entirely
- ✓ Zero-shot means no worked examples are given, though task instructions can still be detailed; the model has no example demonstrations to pattern-match against
- Zero-shot only applies to classification tasks, never open-ended generation
- Zero-shot means the system prompt must be empty

Correct

Zero-shot refers to the absence of example demonstrations, not the absence of instructions; you can still give detailed task instructions with zero examples. It does not require an empty system prompt, does not ignore the question, and applies beyond classification.

A single large system prompt is followed by a per-user profile block that changes daily, then the user's question. You want to cache as much as possible without caching stale profile data. What's the right move?

- Cache the whole assembled prompt as one block, accepting that the profile section forces a full cache miss every time it changes
- Move the profile block after the user's question so it no longer matters where it sits
- Disable caching entirely since any dynamic content anywhere in the prompt invalidates the whole mechanism
- ✓ Place a cache breakpoint after the stable system prompt and a separate one after the profile block, so each stable segment is cached independently

Correct

Multiple cache breakpoints let you cache each stable segment independently, so the daily profile change only invalidates its own segment, not the whole prefix. Caching as one block or disabling it entirely wastes the win available from the stable system prompt, and moving the profile block after the question does not by itself fix cache segmentation.